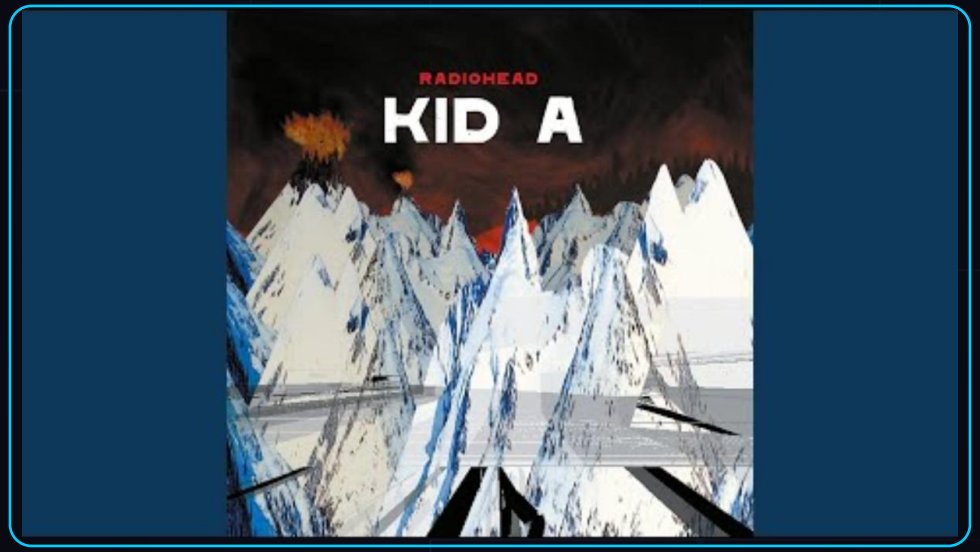
# Architecting with AI

## From Requirements to Reality




**Chad Green**
Senior Systems Architect
Jasper Engines & Transmissions



# THANK YOU to Our Sponsors

## PLATINUM

Progress®

BizStream

meijer®

## GOLD

ATOMIC OBJECT

## SILVER

THE C2 GROUP

COLLEGE OF COMPUTING
GRAND VALLEY STATE UNIVERSITY®

Kentico

uno PLATFORM

LeadingEDJE

AuthorizationHub.

LAFLEUR®

# Architecting with AI

## From Requirements to Reality

*AI doesn't replace architects — it amplifies them.*

**Chad Green**
Senior Systems Architect
Jasper Engines & Transmissions



“I asked AI to write my conference bio.
It called me a *visionary thought leader*
who *disrupts paradigms at scale*.
So… I left it.”

---

Because honestly? Who’s going to argue with that?

A WORD OF CAUTION

**AI is confident. AI is fast.**

AI will absolutely write something that sounds incredible

**and is completely disconnected from reality.**

*That's true for bios. And it's very, very true for architecture.*

## THESIS STATEMENT

> The best architects I know use AI not to replace their thinking, but to accelerate their judgment.

Okay. Now let's ***actually*** talk about how to do that.

# Meet the Speaker



## Chad Green

Senior Systems Architect

**Jasper Engines & Transmissions**

35+ YEARS | SOFTWARE ARCHITECT | SPEAKER

## ABOUT CHAD

- Software architect with **35+ years** building systems that had to **actually work** in production
- Passionate about making **complex architecture** accessible and **fun**
- Early adopter of AI-assisted development — I've seen **the good, the bad, and the hallucinated**
- Conference speaker, community builder, **occasional over-prompter**
- **Superpower:** Turning **"we need a system that does everything"** into something buildable

# Agenda



ACT I

**Requirements Are Lying to You**

*Clarifying requirements with AI, surfacing hidden constraints*



ACT II

## Design Options, Tradeoffs & Adversarial Insight

*Exploring architecture options, stress-testing designs*



ACT III

**Documentation, Governance & Human Judgment**

*AI-assisted documentation, avoiding AI slop, human curation*

CHAPTER ONE

# Act I

## Requirements Are Lying to You

*Why requirements are never as clear as they look.*



# Requirements Hide Landmines

**Ambiguity**
Requirements that mean different things to different people

**Contradictions**
Conflicting rules nobody noticed during review

**Missing NFRs**
The performance, security, and reliability expectations that are never written down

**Hidden Constraints**
Business rules and platform limits that only appear after key decisions are made

# Why Humans Miss Things

## Cognitive Load

Juggling business rules, edge cases, constraints, dependencies, timelines, and capacity — simultaneously — is impossible.

## Anchoring Bias

We latch onto the first interpretation that makes sense and stop exploring alternatives.

## Incomplete Mental Models

We can't ask about constraints we don't realize exist.

- *AI doesn't get tired, biased, or anchored.*

# AI as a Requirements Clarifier

Raw Requirements → AI Analysis → Structured Insights

### Extracts ambiguities

Surfaces the hidden meanings in vague language before they become costly assumptions

### Surfaces hidden NFRs

Infers the performance and security requirements nobody wrote down — but everyone expects

### Identifies contradictions

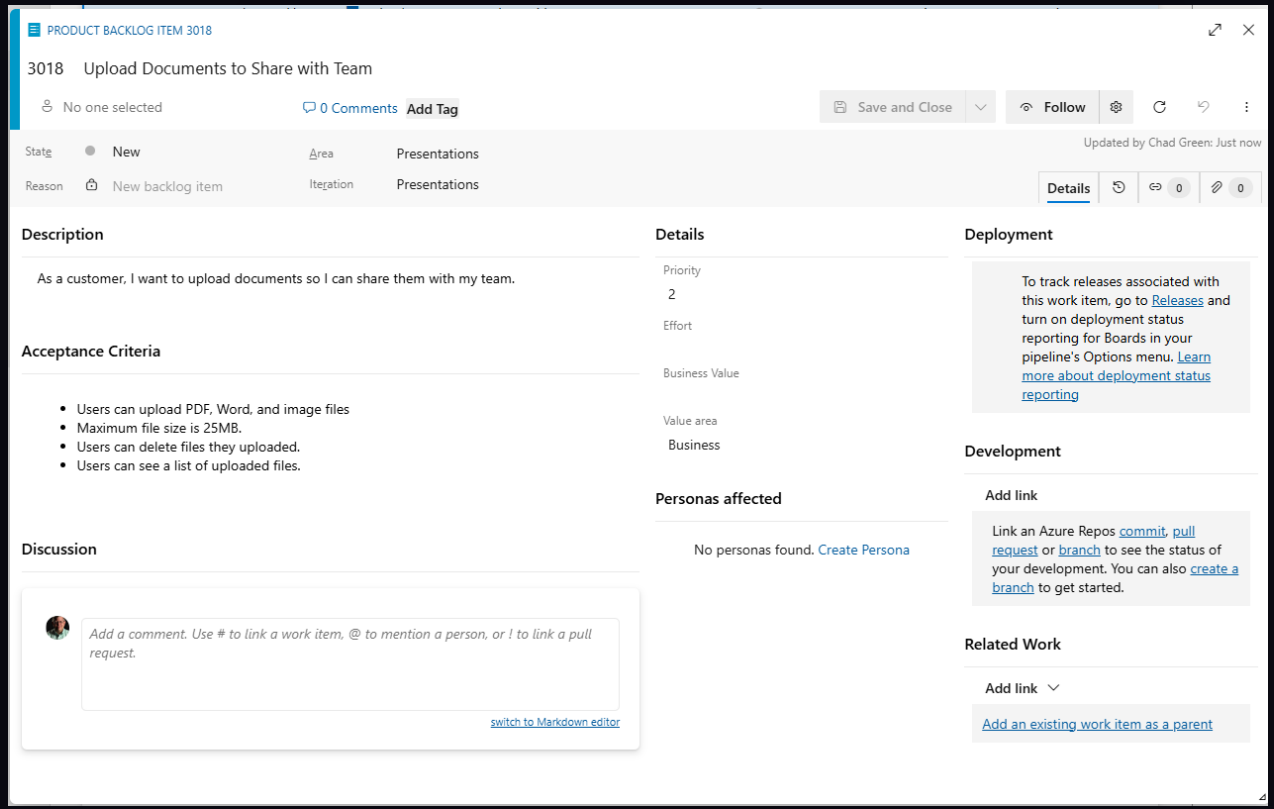
Flags rules that conflict with each other — the ones that survive every review until they break production

### Asks clarifying questions

The ones we forget to ask — precisely when it matters most, before design decisions are locked in

***AI expands awareness, not assumptions.***

# Demo Setup: Requirements Clarification



# Demo Setup: Requirements Clarification

## What You'll See

**AI analyzing user stories**
Ingesting raw requirements as written

**AI surfacing ambiguities & contradictions**
Hidden meanings and conflicting rules exposed

**AI identifying missing NFRs**
Performance, security & reliability gaps

**AI generating clarifying questions**
The ones the team forgot to ask

## The Prompt

copilot-prompt.txt

```
> Analyze the following user story and acceptance
criteria. Identify:

- Ambiguities
- Contradictions
- Missing acceptance criteria
- Implicit non-functional requirements
- Hidden constraints
- Assumptions the stories made
- Questions an architect should ask before designing
  a solution

DO NOT invent new features. Only analyze what is
present or implied.

Ask clarifying questions as you work.
```

# LIVE DEMO 1

## Requirements Analysis with Copilot

# What AI Got Right / What AI Got Wrong

## What It Got Right

- Surfaced real ambiguities in the requirements
- Identified missing acceptance criteria no one had flagged
- Pointed out hidden NFRs that matter in production
- Exposed contradictions nobody had noticed

vs

## What It Might Get Wrong

- Hallucinated constraints that didn't exist
- Made assumptions not present in the requirements
- Confidently stated incorrect conclusions
- Over-applied patterns from unrelated domains

*AI expands the problem space — and the nonsense space. Our job is to separate the two.*



AI expands the problem space —
but architects decide what’s real.

AI helps you see more. AI helps you ask better questions.
The judgment is still yours.

SECTION II OF III

# Act II

## Design Options, Tradeoffs & Adversarial Insight

Exploring more possibilities, faster —
then stress-testing everything.



AI WIDENS THE SOLUTION SPACE

# AI for Architecture Exploration

- **Generate 4–5 viable architecture options in seconds** — not just the obvious one
- **Compare tradeoffs** across reliability, scalability, cost, and complexity
- **Evaluate quality attributes** — performance, security, maintainability, observability
- **Explore architectural patterns** you might not have considered

*AI doesn't anchor. That's the point.*

# Demo Setup: Architecture Options

## What You'll See

- AI proposing 4 architecture approaches
- AI comparing strengths and weaknesses
- AI identifying risks per option
- AI surfacing hidden assumptions

## The Prompt

copilot-prompt.txt

```
> Given the clarified requirements, propose 4
  distinct architecture approaches:

  - Modular monolith
  - Microservices
  - Event-driven
  - Serverless

  For each, include:

  - High-level description
  - Strengths
  - Weaknesses
  - Key risks
  - Team skill assumptions
```

# LIVE DEMO 2

## Architecture Options + Tradeoffs

*Switching to IDE…*

# The Power of Adversarial AI

After choosing a preferred architecture — stress-test it.

1

### Different Model

Use a separate AI perspective to avoid confirmation bias.

2

### Different Perspective

Explicitly instruct it to attack, not validate, your design.

3

### Brutally Honest

It will surface risks, gaps, and failure modes without sugarcoating.

4

### Finds Blind Spots

Hidden assumptions, scaling risks, security gaps, data consistency issues.

⚠ ***The closest thing to a pre-mortem without deploying to production.***

# Demo Setup: Adversarial Agent

## What You'll See

- AI attacking the chosen architecture
- AI identifying hidden assumptions
- AI surfacing scaling and operational risks
- AI predicting likely failure modes

## The Prompt

copilot-prompt.txt

```
> Given the clarified requirements, propose 4
  distinct architecture approaches:

  - Modular monolith
  - Microservices
  - Event-driven
  - Serverless

  For each, include:

  - High-level description
  - Strengths
  - Weaknesses
  - Key risks
  - Team skill assumptions
```

# LIVE DEMO 3

## Adversarial Review

*Switching to IDE…*

ACT II — KEY TAKEAWAY



# AI challenges us in ways humans rarely do — and that makes our designs stronger.

AI gives you breadth. Adversarial AI gives you depth.

Together they build more resilient, defensible architectures.

# Act III

## Documentation, Governance & Human Judgment

*Where AI ends and human judgment begins.*

⚠ CAUTION

# The Danger of AI Slop

### Volume Without Clarity

AI can generate **too much** — volume without clarity is noise, not documentation

### Teams Drown in Docs

When **everything** is documented, **nothing** is documented

### Architects Must Curate

We decide what **matters**, what **belongs**, and what gets **cut**

⚠ ***AI is a drafting engine — not a decision-maker.***

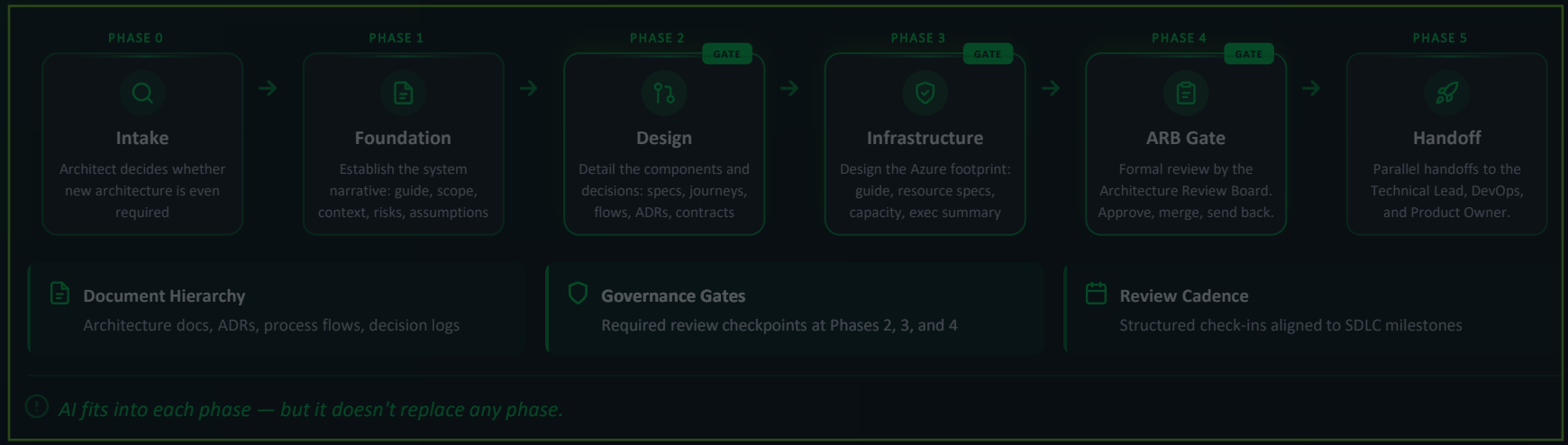
# The Jasper Architecture Process




## Document Hierarchy

Architecture docs, ADRs, process flows, decision logs

# The Jasper Architecture Process




## Governance Gates

Required review checkpoints at Phases 2, 3, and 4

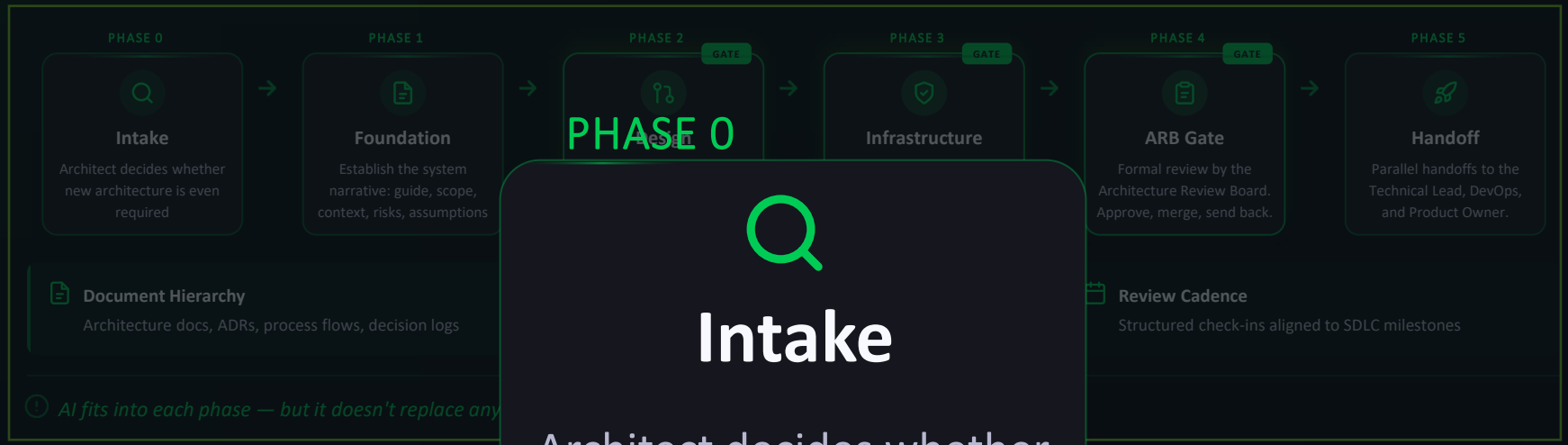
# The Jasper Architecture Process




## Review Cadence

Structured check-ins aligned to SDLC milestones

# The Jasper Architecture Process

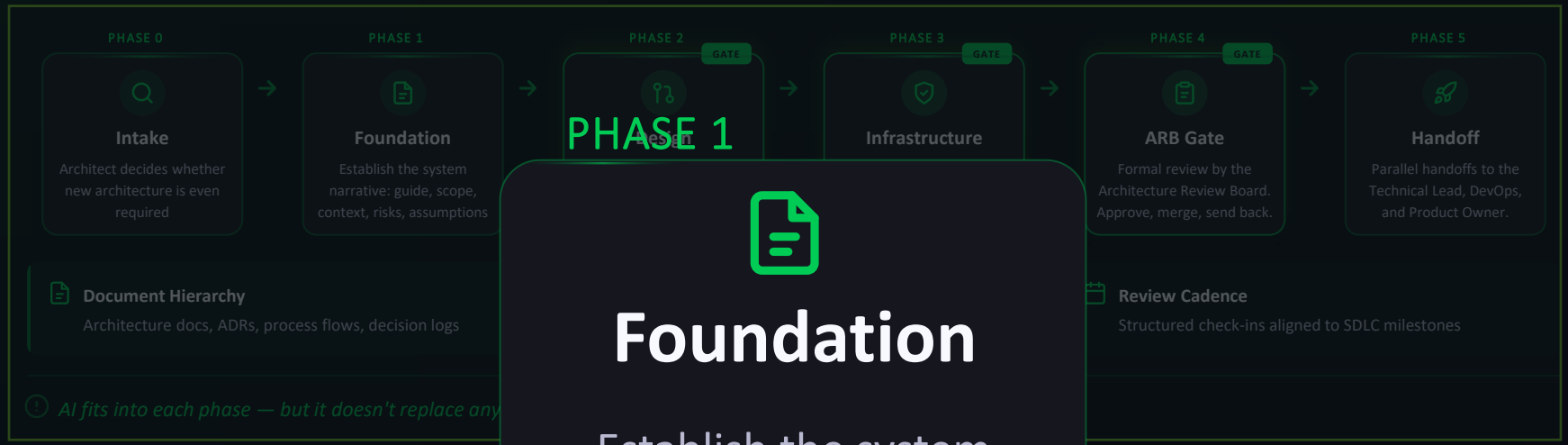



PHASE 0

## Intake

Architect decides whether new architecture is event required.

# The Jasper Architecture Process

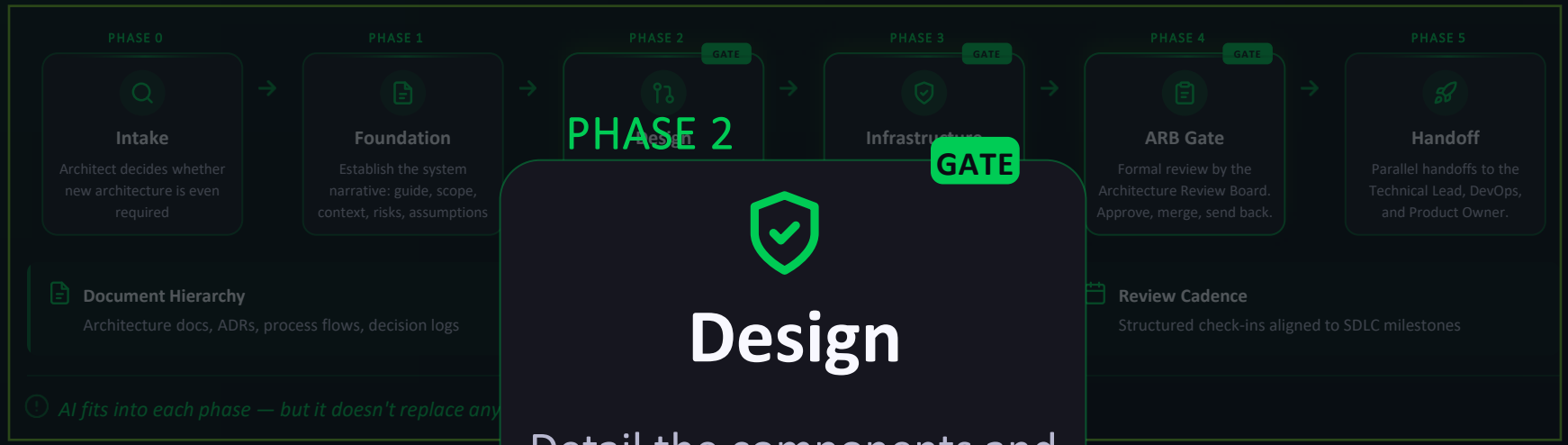



PHASE 1

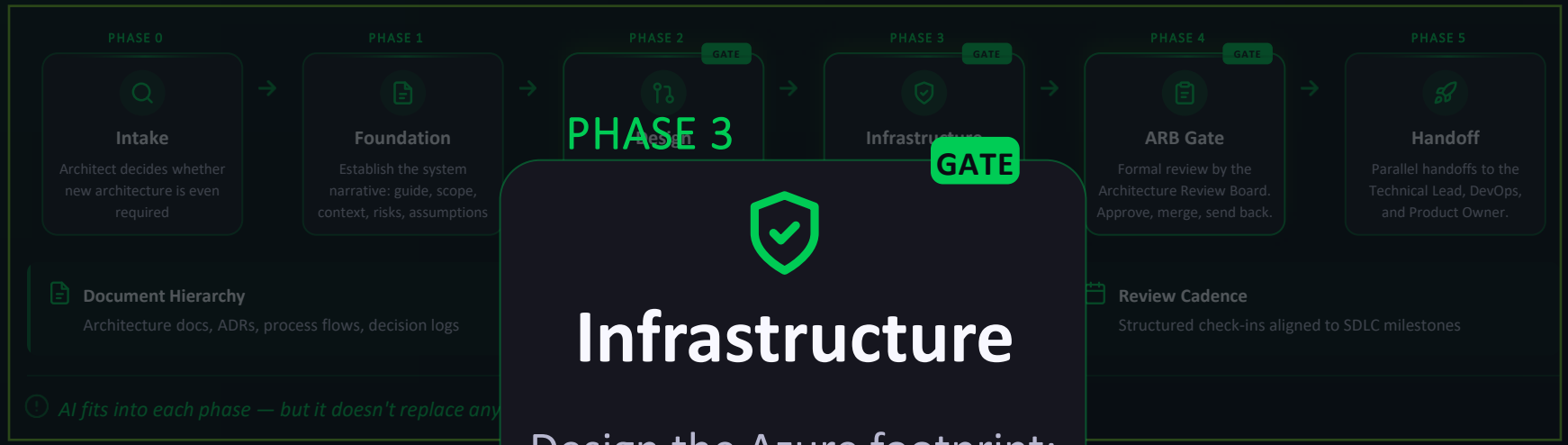
## Foundation

Establish the system narrative: guide, scope, context, risks, assumptions.

# The Jasper Architecture Process



# The Jasper Architecture Process



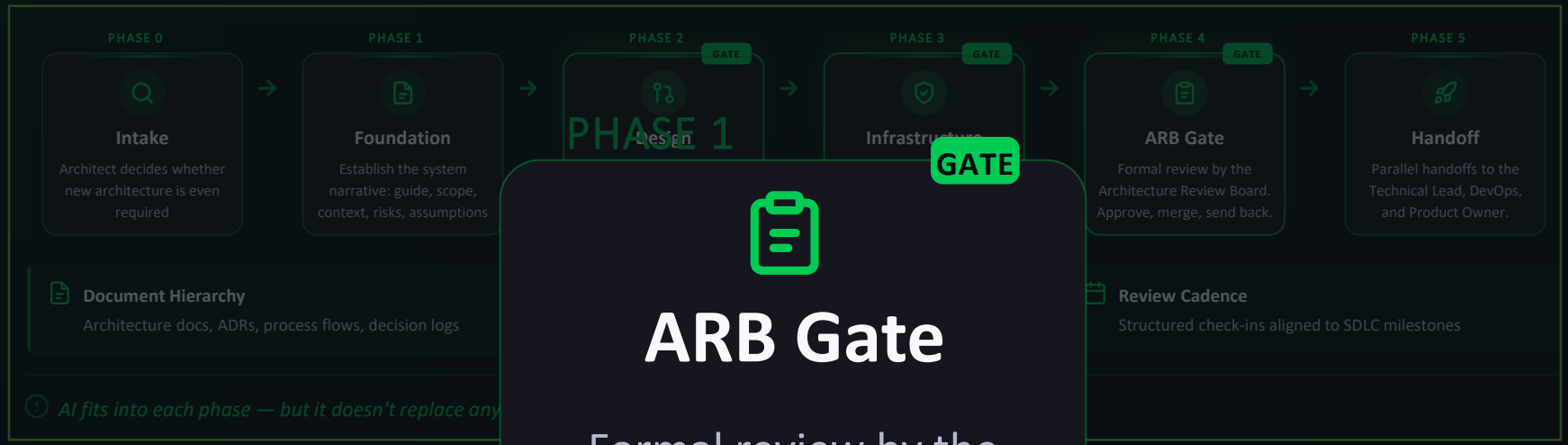

PHASE 3

GATE

## Infrastructure

Design the Azure footprint: guide, resource specs, capacity, exec summary.

# The Jasper Architecture Process

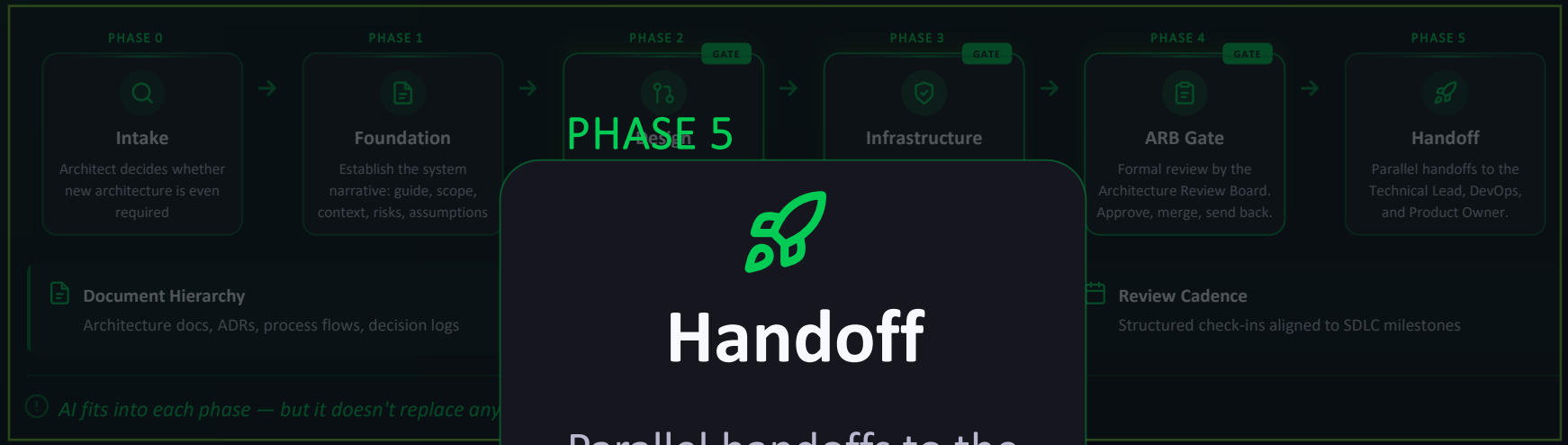



GATE

## ARB Gate

Formal review by the Architecture Review Board. Approve, merge, send back.

# The Jasper Architecture Process




PHASE 5

## Handoff

Parallel handoffs to the Technical Lead, DevOps, and Product Owner.

# The Jasper Architecture Process




**Document Hierarchy**
Architecture docs, ADRs, process flows, decision logs

**Governance Gates**
Required review checkpoints at Phases 2, 3, and 4

**Review Cadence**
Structured check-ins aligned to SDLC milestones

*AI fits into each phase — but it doesn't replace any phase.*

# Templates + Instructions + Examples = Quality

**Structure** → **Clarity** → **Quality**

### Prevent AI Slop

Constrain → Focus

### Enforce Structure

Ground → Govern

### Accelerate Drafting

Speed → Quality

**"AI wrote this"** vs. **"AI helped me write this"** — the difference is **structure.**

# Demo Setup: Document Generation

## What You'll See

- ADR generation using a structured template
- Process flow generation from an architecture decision
- How instructions enforce quality and prevent drift
- How unconstrained AI produces slop — contrast example

Structure + instructions = quality output — every time.

## The Prompts

ADR PROMPT — TEMPLATE-BOUND

```
> Using the ADR template, instructions, and example (located at
docs-templates/adr), generate an ADR for the decision to use a
[selected architecture].
```

PROCESS FLOW PROMPT — TEMPLATE-BOUND

```
> Using the Process Flow template, instructions, and example
(located at docs-templates/process-flow), generate a process flow
document for the file upload workflow.
```

# LIVE DEMO 4

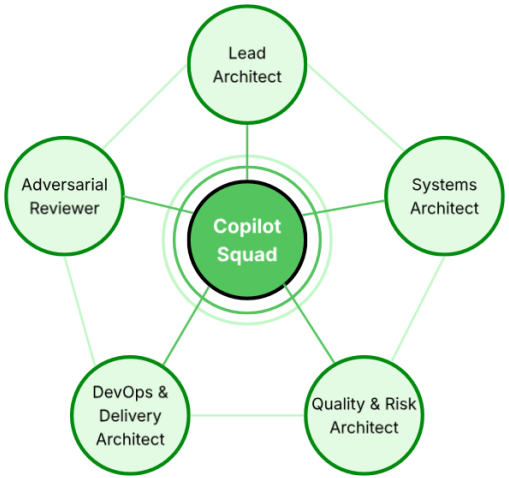
## ADR + Process Flow Generation

*Switching to IDE…*

# Copilot Squad

PREVIEW ONLY

## Multi-Agent Architectural Workflows




### Multi-agent workflows

Specialized AI agents working in parallel — not one model doing everything

### Each agent has a distinct role and defined scope

Specialization prevents context bleed and hallucination drift

### No single agent holds too much context

Bounded scope keeps each agent focused, accurate, and auditable

### Together, they mirror a full architecture review team

Requirements → Design → Tradeoffs → Adversarial Review → Documentation

*Seek me out, and I talk all day about Squad.*

ACT III — KEY TAKEAWAY

“

# AI accelerates drafting — but architects ensure correctness.

AI helps you write faster. AI helps you structure your thinking.
But the judgment, the decisions, the governance — that's still ours.

# Closing

## Bringing It All Together

*Where AI ends and architects begin.*

# The New Role of the Architect

01

### AI Expands Possibilities

More options, more patterns, more tradeoffs, more risks explored — faster than ever before.

02

### Humans Provide Judgment

We decide what is real, what is feasible, what is responsible — and we own the consequences.

03

### The Partnership Matters

Human judgment plus AI acceleration — not replacement, not automation. Augmentation.

***Not replacement. Not automation. Augmentation.***

CLOSING STATEMENT

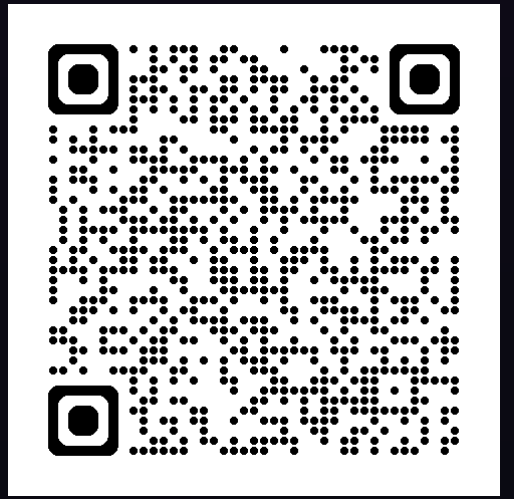
# You now have superpowers.

Use them to build better systems, better teams,
and better futures.

*Because AI doesn't replace architects — AI amplifies architects.*

# Thank You

## Q & A




**Chad Green**

Senior Systems Architect — Jasper Engines & Transmissions

linkedin.com/in/chadwickgreen  github.com/talelearncode  chadgreen@chadgreen.com